

Phison LobeChat User Guide

Overview

Welcome to the Phison LobeChat User Guide! This comprehensive guide will walk you through the installation, setup, and usage of Phison's integration with LobeChat.

Phison LobeChat is an AI-powered roleplay interface that leverages the innovative **aiDAPTIV+** KV Cache technology to provide lightning-fast responses to your questions. By building a knowledge cache of your chat context, it enables near-instantaneous answers and significantly enhances your experience.

This guide covers:

- Understanding the aiDAPTIV+ KV Cache feature
- How to use the application with demo examples
- Installation and setup procedures
- Troubleshooting common issues

Let's get started!

1. Installation

We have provided automated PowerShell scripts to streamline the installation process.

Step 1: Clone the Repository

Open your terminal (PowerShell recommended) and clone the project:

```
git clone <repository-url>
cd aiDAPTIV-Integration-lobe-chat
```

Step 2: Install Prerequisites

Run the first installer script to set up Node.js and other necessary tools.

```
./installer/1.install_prerequisites.ps1
```

Note: You may need to run PowerShell as Administrator.

Step 3: Setup Project

Run the second script to install dependencies and configure the environment.

```
./installer/2.setup_project.ps1
```

2. Prerequisites

Before installing, ensure your system meets the following requirements:

- **Docker Desktop:** Required for running local databases and services
- **Node.js:** Version 22 or higher (Included in the installer scripts)

3. Feature Showcase: aiDAPTIV+ & Demo

The core feature of this integration is the **Automatic KV Cache Warming**. This ensures that when you interact with an AI agent, the context is pre-processed, resulting in faster response times.

Demo Scenario: Sherlock Holmes (Arthur Conan Doyle)

We have provided a demo session file to showcase the capabilities.

1. Start with Demo Mode (Recommended)

We have simplified the process with a one-click script that automatically sets up the environment and loads the character settings.

1. Navigate to the `Example` folder in the project directory.
2. Double-click on `Demo_start.bat`.
3. This script will:
 - Start the development server.
 - Automatically create a new chat session.
 - Load the "Sherlock Holmes" character configuration (System Role & Opening Message).
 - **Automatically trigger the KV Cache Warming** immediately upon creation.

(Alternatively, you can still manually import `Example/LobeChat-Sir-Arthur-Conan-Doyle-session-v7.json` via the UI if running in standard mode.)

2. Triggering the KV Cache Build (Manual)

If you want to see the KV Cache build process in action manually (or if you modify the character):

1. Click on the **Agent Settings** (usually a gear icon or the agent's avatar).
2. Navigate to the **Prompt** tab (System Role settings).
3. Make a small edit to the **System Role** text (e.g., add a space or change a word).
4. **Wait for few seconds.**
 - The system automatically detects the change.
 - It sends a background request to the local AI model to "warm up" the KV Cache.

3. Experience the Speed

Once the cache is built:

1. Return to the chat window.
2. Ask a complex question related to the character.
3. Notice that the **Time to First Token (TTFT)** is significantly reduced compared to a standard cold start.

4. Running the Application

Once the installation is complete, you have two options to start the application.

Option A: Demo Mode (Recommended for Showcase)

1. Go to the `Example` folder.
2. Run `Demo_start.bat`.
3. This script will:
 - Check and start Docker Desktop if needed.
 - Start necessary backend services (Database, etc.).
 - Launch the application with the Sherlock Holmes character pre-loaded.

Option B: Standard Development Mode

1. Go to the project root directory.
2. Run `Start_Dev.bat`.
3. This script will:
 - Check and start Docker Desktop if needed.
 - Start necessary backend services.
 - Launch the development server at <http://localhost:3010>.

5. Configuration

Model Provider Settings

By default, the application is configured to connect to a local LLM endpoint (e.g., `llama.cpp` running on your network or localhost).

To verify or change this:

1. Go to **Settings** -> **Language Models**.
2. Check the **OpenAI** provider settings.
3. Ensure the **Proxy URL** is pointing to your local aiDAPTIV+ server (e.g., `http://localhost:13141/v1`).

6. Troubleshooting

Q: I get a 401 Unauthorized error in the console when editing settings. A: This issue has been resolved in the latest update. Ensure you have pulled the latest code. The system now correctly handles authentication headers for local proxy requests.

Q: The application is not starting. A: Ensure Docker Desktop is running, as the local database requires it. Run `docker ps` to verify containers are active.

Q: The KV Cache doesn't seem to trigger. A: The trigger has a few second debounce timer. Make sure you stop typing for at least 10 seconds. Also, ensure you are

editing the *System Role* field, as this is what defines the agent's core context.
